Nebius Token Factory + OpenClaw — Test Report

> **Result: working.** A Nebius Token Factory API key drove OpenClaw end to end: model listing, chat, tool calling, image input, and a full agent turn through the OpenClaw gateway, all on Nemotron.

Tested on 2026-10-27 on the isaac-claw host (Linux, OpenClaw 2026.5.7 gateway as a systemd user service on port 18789, Python 3.11, `openai` SDK 3.40.0). Setup steps are in `../NEBIUS.md`.

Summary

Check	Result
Key accepted, models listed	✅ 5 Nemotron models visible
Nemotron 3 Super — chat	✅ answers (needs `max_tokens` ≥ 1024, see below)
Nemotron 3 Super — tool calling	✅ correct `tool_calls`
Nemotron 3 Nano — chat	✅ 0.7 s
Nemotron 3.5 Super VL — chat / tools / vision	✅ on both global and `us-north1` endpoints
OpenClaw gateway starts on Nebius	✅ `agent model: nebius/nvidia/Nemotron-3.5-Super-VL`
OpenClaw agent turn via gateway	✅ replied in ~10 s, used isaac-claw skills
NemoClaw sandbox on Nebius	⏸ not tested yet

Models available to the key

Endpoint: `https://api.tokenfactory.nebius.com/v1` (OpenAI-compatible).

Model ID	Context	Input	Price in / out (per 1M tokens)	Rate limit	Region
`nvidia/Nemotron-3.5-Super-VL`	1,048,576	text + image	$0.30 / $0.90	2M tok/min, 300 req/min	us-north1
`nvidia/nemotron-3-super-120b-a12b`	262,144	text	$0.30 / $0.90	200K tok/min, 300 req/min	us-central1
`nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B`	262,144	text	$0.06 / $0.24	800K tok/min, 100 req/min	eu-north1
`nvidia/Nemotron-3-Ultra-550b-a55b`	—	text	—	—	—
`nvidia/Nemotron-3_5-Lightning`	—	—	—	—	—

All three tested models report `tools` and `reasoning` support. Ultra and Lightning were listed but not exercised.

Test details

1. `scripts/test_nebius.py`

Model	Time	Tokens	Answer
Nemotron 3 Super	1.2 s	247	*(empty — see note)*
Nemotron 3 Nano	0.7 s	160	“Use the simulator’s terrain generator to create a curriculum of randomized, uneven surfaces and immediately begin reinforcement-learning trials that reward stable, energy-efficient forward gaits.”

Note: Super is a reasoning model. At `max_tokens=200` it spent the whole budget reasoning (`finish_reason=length`, 0 chars of answer). At `max_tokens=1500` it finished normally (`finish_reason=stop`, 494 completion tokens) with a proper answer. The script now uses `max_tokens=1024`.

2. Tool calling (Nemotron 3 Super)

Prompt: *“Start training the G1 rough-terrain locomotion task with 4096 envs.”* with one `launch_training(task, num_envs)` tool offered.

```
launch_training({"task": "G1 rough-terrain locomotion", "num_envs": 4096})   finish_reason=tool_calls
```

3. Nemotron 3.5 Super VL — both endpoints

Endpoint	Chat	Tool call	Vision (solid red image)
`api.tokenfactory.us-north1.nebius.com/v1/`	5.3 s, 675 tok	1.1 s ✅	0.8 s → “Red” ✅
`api.tokenfactory.nebius.com/v1`	4.0 s, 525 tok	1.3 s ✅	1.0 s → “Red” ✅

Both endpoints work with the same key. The global endpoint is used in the configs because the sandbox egress allowlist already covers it.

4. OpenClaw gateway + agent turn

Gateway log after restart:

```
[gateway] agent model: nebius/nvidia/Nemotron-3.5-Super-VL (thinking=medium, fast=off)
[gateway] http server listening (7 plugins: ...; 3.1s)
```

Agent turn (isolated session, so the `main` chat history is untouched):

```
openclaw agent --agent main --session-id nebius-smoke-test --timeout 200 \
  -m "Smoke test, do not use any tools. In two short sentences: say which model you are running on, then
what you would do first to train a Unitree G1 to walk on rough terrain in Isaac Lab."
```

Reply (~9.7 s total):

> I’m running on the nebius/nvidia/Nemotron-3.5-Super-VL model. First I’d profile the Unitree G1 embodiment with robot-profile build, then set up the Isaac Lab rough-terrain task and train via policy-trainer.

The agent named the right model and picked the isaac-claw `robot-profile` and `policy-trainer` skills on its own.

OpenClaw config that was verified

Added to `~/.openclaw/openclaw.json` (everything else left as it was):

```
{
  "models": {
    "providers": {
      "nebius": {
        "baseUrl": "https://api.tokenfactory.nebius.com/v1",
        "apiKey": "${NEBIUS_API_KEY}",
        "api": "openai-completions",
        "models": [
          { "id": "nvidia/Nemotron-3.5-Super-VL", "name": "Nemotron 3.5 Super VL (Nebius)",
            "reasoning": true, "input": ["text", "image"], "contextWindow": 262144, "maxTokens": 8192 },
          { "id": "nvidia/nemotron-3-super-120b-a12b", "name": "Nemotron 3 Super (Nebius)",
            "reasoning": true, "contextWindow": 262144, "maxTokens": 8192 },
          { "id": "nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B", "name": "Nemotron 3 Nano (Nebius)",
            "reasoning": true, "contextWindow": 262144, "maxTokens": 8192 }
        ]
      }
    }
  },
  "agents": {
    "defaults": {
      "model": {
        "primary": "nebius/nvidia/Nemotron-3.5-Super-VL",
        "fallbacks": [
          "nebius/nvidia/nemotron-3-super-120b-a12b",
          "nebius/nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B"
        ]
      }
    }
  }
}
```

`contextWindow` is capped at 256K to keep long sessions cheap; 3.5 Super VL supports 1M. The repo’s `agents/openclaw/model.json` still uses Nemotron 3 Super as primary. Both models passed.

How to reproduce

```
# 1. Key — kept in OpenClaw's env file so every OpenClaw process sees it
read -s "k?Nebius key: " && echo "NEBIUS_API_KEY=$k" >> ~/.openclaw/.env && chmod 600 ~/.openclaw/.env;
unset k

# 2. API check
python3 -m venv .venv && .venv/bin/pip install openai
set -a && . ~/.openclaw/.env && set +a && .venv/bin/python scripts/test_nebius.py

# 3. OpenClaw: back up, add the block above, validate, restart
cp -p ~/.openclaw/openclaw.json ~/.openclaw/openclaw.json.pre-nebius
#    ...edit ~/.openclaw/openclaw.json...
openclaw config validate
systemctl --user restart openclaw-gateway

# 4. Agent turn
openclaw agent --agent main --session-id nebius-smoke-test -m "Which model are you running on?"

# 5. Use it: launch the TUI the repo way (not bare `openclaw tui`)
agents/openclaw/tui.sh
```

Gotchas found

- **Missing key breaks all of OpenClaw.** With `${NEBIUS_API_KEY}` in the config, OpenClaw refuses to load the config at all if the variable is unset — the local fallbacks don't help. Keep the key in `~/.openclaw/.env` and restart the gateway after adding it (a running gateway won't see a new `.env`).
- `openclaw config patch` **false failure.** It rejected unrelated WhatsApp fields (`debounceMs`, `mediaMaxMb`) even though `openclaw config validate` passed on the same config. Editing the file directly and then running `openclaw config validate` works.
- **The gateway copies the resolved key** into `~/.openclaw/agents/main/agent/models.json`. Remove the `nebius` entry there too when removing or rotating the key.
- **Reasoning eats** `max_tokens`**.** Give Super at least ~1024 tokens or answers come back empty.
- **WhatsApp.** If the WhatsApp channel is enabled, its chats with the `main` agent go to Nebius too, and are billed.

Rollback

```
systemctl --user stop openclaw-gateway
cp ~/.openclaw/openclaw.json.pre-nebius ~/.openclaw/openclaw.json
rm ~/.openclaw/.env          # and remove "nebius" from ~/.openclaw/agents/main/agent/models.json
```

Not done yet

- **NemoClaw sandbox on Nebius** — not onboarded or tested.
- **Allowlists** — `agents/nemoclaw/agent/isaac-orchestrator/policy-additions.yaml` and the `policy` block in `agents/nemoclaw/agent/isaac-orchestrator/blueprint.yaml` don't list `api.tokenfactory.nebius.com` yet (only `sandbox/policies/orchestrator-policy-additions.yaml` does).

State after the test

After testing, the key was removed from the test machine, the OpenClaw config was restored to its pre-Nebius version, and the gateway was stopped. To run on Nebius again, create a fresh key in Token Factory and follow *How to reproduce*.